

Árboles en Java

Vamos a generar un resumen conceptual para alumnos de árbol general, árbol binario, árbol BST (incluyendo recorridos LIFO y FIFO, es decir DFS y BFS) y árboles balanceados (AVL y RED-BLACK). Asimismo introducimos la idea de la interfaz SET y su implementación en un TreeSet. Luego vamos a generar un detalle de la actividad hecha donde organizabamos archivos en un árbol BST casero y luego hicimos el mismo ejemplo con TreeSet. Y finalmente la explicación de la actividad hecha de árbol general que simula el explorador de Windows. Dejamos lo demás para la clase que viene (implementación de un árbol que lee realmente la estructura de carpetas y buscar archivos y los lee).

Unidad: Árboles en Java – Resumen Conceptual y Actividades Realizadas

1. ¿Qué es un Árbol?

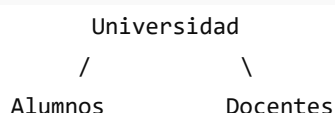
Un **árbol** es una estructura de datos jerárquica formada por nodos.

Cada nodo puede contener información y referencias a otros nodos.

Conceptos básicos:

- **Raíz (Root):** primer nodo del árbol.
- **Nodo Padre:** nodo que contiene referencias a otros nodos.
- **Nodo Hijo:** nodo dependiente de otro.
- **Hoja:** nodo sin hijos.
- **Nivel:** profundidad dentro del árbol.
- **Subárbol:** árbol contenido dentro de otro árbol.

Ejemplo:



/ \ |
Ana Juan Carlos

2. Árbol General

Un **árbol general** es aquel donde un nodo puede tener cualquier cantidad de hijos.

Ejemplo:

```
C:
├── Documentos
│   ├── Trabajo.docx
│   └── TP.pdf
├── Imágenes
│   ├── Foto1.jpg
│   ├── Foto2.jpg
│   └── Foto3.jpg
└── Música
```

No existe límite de hijos por nodo.

Se utiliza para modelar:

- Sistemas de archivos.
- Organigramas.
- Árboles genealógicos.
- Menús de aplicaciones.

3. Árbol Binario

Un **árbol binario** es un caso particular donde cada nodo puede tener como máximo dos hijos:

- Hijo izquierdo.
- Hijo derecho.

Ejemplo:

```
      A
     / \
    B   C
   / \
  D   E
```

No existe ninguna regla de orden.

Solamente se limita a tener dos ramas por nodo.

4. Árbol Binario de Búsqueda (BST)

BST significa:

Binary Search Tree

Es un árbol binario que agrega una regla fundamental:

Izquierda < Nodo < Derecha

Ejemplo:

Insertando:

50, 30, 70, 20, 40, 60, 80

obtenemos:

```
      50
     /  \
    30   70
   / \  / \
  20 40 60 80
```

Ventajas

Permite:

- Insertar rápidamente.
- Buscar rápidamente.
- Mantener los datos ordenados.

5. Recorridos de un BST

Los recorridos permiten visitar todos los nodos del árbol.

DFS (Depth First Search)

DFS significa:

Búsqueda en Profundidad

Explora primero una rama completa y luego vuelve atrás.

Se implementa naturalmente mediante:

- Recursividad
- Pila (Stack)

Por eso suele asociarse con comportamiento:

LIFO
Last In First Out

PreOrden

Nodo
Izquierda
Derecha

Ejemplo:

```
  50
 /  \
30   70
```

Resultado:

50 30 70

InOrden

Izquierda
Nodo
Derecha

Resultado:

```
30 50 70
```

Particularidad:

En un BST devuelve los elementos ordenados.

PostOrden

```
Izquierda  
Derecha  
Nodo
```

Resultado:

```
30 70 50
```

6. BFS (Breadth First Search)

BFS significa:

Búsqueda por Amplitud

Recorre nivel por nivel.

Ejemplo:

```
      50  
     /  \  
    30   70  
   /  \  
  20 40 60 80
```

Resultado:

```
50  
30 70  
20 40 60 80
```

O:

```
50 30 70 20 40 60 80
```

¿Cómo se implementa?

Con una cola.

```
FIFO  
First In First Out
```

Por eso BFS suele asociarse a:

```
</> Java  
Queue
```

7. Problema de los BST

Los BST pueden desbalancearse.

Ejemplo:

Insertando:

```
10  
20  
30  
40  
50
```

obtenemos:

```
10  
 \  
 20  
  \  
  30  
   \  
    40  
     \  
      50
```

Prácticamente se convierte en una lista.

La búsqueda deja de ser eficiente.

8. Árboles Balanceados

Para solucionar ese problema existen árboles auto-balanceados.

AVL

Creado por:

- Adelson-Velsky
- Landis

Características:

- Mantiene equilibradas ambas ramas.
- Diferencia de altura máxima:

```
-1
0
+1
```

Si se rompe esa condición:

- realiza rotaciones.

Ejemplo:

```
30
 /
20
 /
10
```

Se rota para quedar:

```
  20
 /  \
10   30
```

Ventaja:

- Búsquedas extremadamente rápidas.

Desventaja:

- Más rotaciones.

Red-Black Tree

Cada nodo posee un color:

Rojos
Negros

Reglas especiales garantizan que el árbol permanezca aproximadamente balanceado.

Ventajas:

- Menos rotaciones que AVL.
- Muy eficiente para inserciones y eliminaciones.

9. ¿Cuál utiliza Java?

La implementación interna de:

</> Java
TreeSet
TreeMap

utiliza un:

Red-Black Tree

Por eso:

</> Java
TreeSet

mantiene automáticamente:

- Ordenamiento.
- Balanceo.
- Búsquedas eficientes.

10. La Interfaz Set

La interfaz:


```
</> Java
```

```
Set<E>
```

representa una colección sin elementos repetidos.

Ejemplo:

```
</> Java
```

```
Set<String> nombres = new HashSet<>();
```

```
nombres.add("Ana");
```

```
nombres.add("Ana");
```

```
nombres.add("Juan");
```

Resultado:

Ana

Juan

No admite duplicados.

11. TreeSet

Implementación de:

```
</> Java
```

```
Set<E>
```

basada en Red-Black Tree.

```
</> Java
```

```
TreeSet<String> archivos = new TreeSet<>();
```

Características:

- ✓ No admite repetidos
- ✓ Mantiene ordenados los datos
- ✓ Balanceado automáticamente
- ✓ Búsquedas rápidas

Actividad Realizada 1

Organización de archivos usando BST Casero

Objetivo:

Representar una carpeta con varios archivos utilizando un BST implementado por nosotros.

Datos cargados

Ejemplo:

```
informe.pdf
foto.jpg
tp.docx
notas.txt
resumen.pdf
java.pdf
ejercicio.docx
```

Inserción

Cada archivo se inserta comparando su nombre.

```
</> Java
compareTo()
```

determina:

- izquierda
- derecha

siguiendo la regla:

```
Izquierda < Nodo < Derecha
```

Recorridos realizados

DFS

Mediante:

- PreOrden
 - InOrden
 - PostOrden
-

BFS

Mediante:

```
</> Java
Queue<Nodo>
```

recorriendo nivel por nivel.

Objetivo pedagógico

Comprender:

- estructura interna de un BST
 - recursividad
 - nodos enlazados
 - DFS
 - BFS
-

Actividad Realizada 2

Organización de archivos usando TreeSet

Luego se resolvió el mismo problema utilizando:

```
</> Java
TreeSet<String>
```

Ejemplo:

```
</> Java
TreeSet<String> archivos = new TreeSet<>();

archivos.add("informe.pdf");
archivos.add("foto.jpg");
archivos.add("tp.docx");
```

Resultado

Automáticamente:

- quedan ordenados.
- no hay duplicados.
- no debemos programar nodos.
- no debemos programar inserciones.
- no debemos programar balanceo.

Conclusión

Nuestro BST casero sirve para comprender cómo funcionan los árboles.

TreeSet es la solución profesional que Java ofrece para resolver el mismo problema de forma eficiente.

Actividad Realizada 3

Árbol General simulando el Explorador de Windows

Objetivo:

Representar la estructura de carpetas de Windows mediante un árbol general.

Ejemplo:

```
C:
├── Documentos
│   ├── TP1
│   ├── TP2
│   └── Parciales
├── Imágenes
│   ├── Vacaciones
│   └── Trabajo
└── Música
```

Implementación

Cada nodo representa:

- una carpeta
- o un archivo

y contiene una lista de hijos:

```
</> Java
```

```
ArrayList<NodoGeneral>
```

Operaciones realizadas

Crear carpetas

```
</> Java
```

```
Documentos
```

```
Imágenes
```

```
Música
```

Agregar subcarpetas

```
</> Java
```

```
TP1
```

```
TP2
```

```
Vacaciones
```

```
Trabajo
```

Mostrar estructura

Salida:

```
C:
```

```
├─ Documentos
```

```
│   ├─ TP1
```

```
│   └─ TP2
```

```
├─ Imágenes
```

```
│   └─ Vacaciones
```

```
│   └─ Trabajo
```

```
└─ Música
```

Objetivo pedagógico

Comprender cómo los sistemas operativos organizan la información.

El Explorador de Windows, Linux y macOS utilizan internamente estructuras jerárquicas equivalentes a árboles generales.

Próxima Clase

En la siguiente etapa abandonaremos los datos cargados manualmente y construiremos un árbol utilizando la estructura real del sistema de archivos mediante:

</> Java

File
FileReader
BufferedReader

Trabajaremos con:

- Lectura real de carpetas.
- Construcción automática del árbol.
- Búsqueda de archivos.
- Lectura del contenido de archivos.
- Recorridos sobre estructuras reales del sistema operativo.

